GUIA TÉCNICO | 05

Funções de perda, gradientes e otimização

Capítulo 05/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Calcular perdas e inspecionar gradientes analíticos e numéricos.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

PROVENIÊNCIA E PRÉ-REQUISITOS

- Material histórico de apoio: Unidades 3 e 4 — fundamentos de backpropagation, gradientes e treinamento, páginas 126–259 do PDF consolidado.
-

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

O objetivo do ajuste é minimizar uma função de perda no conjunto de treino. Para regressão, MSE penaliza erros quadráticos; para classificação binária, log-loss exige probabilidades e favorece previsões calibradas. Taxa de aprendizado grande pode divergir.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: $MSE=(1/n)\Sigma(\hat{y}-y)^2$

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

Verifique as hipóteses matemáticas do exemplo, os formatos das entradas e saídas e a relação entre o problema e o algoritmo de treino.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
x=np.array([1.,2.,3.]);y=np.array([2.,4.,6.]);w=0.
def mse(t): return np.mean((t*x-y)**2)
for _ in range(120):
 grad=2*np.mean((w*x-y)*x)
 w-=0.05*grad
eps=1e-5
num=(mse(w+eps)-mse(w-eps))/(2*eps)
ana=2*np.mean((w*x-y)*x)
print('w otimizado',round(w,5),'perda',round(mse(w),8),'gradiente erro',abs(num-ana))
assert abs(w-2)<1e-3
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um

conjunto final de teste.

04 LABORATÓRIO APLICADO

```
import numpy as np

rng = np.random.default_rng(7)
x = np.linspace(-2., 2., 100)
y = 1.5 + 2.2*x + rng.normal(scale=.15, size=x.size)
indices = rng.permutation(x.size)
tr, te = indices[:80], indices[80:]
Xtr = np.column_stack([np.ones(tr.size), x[tr]])
Xte = np.column_stack([np.ones(te.size), x[te]])
ytr, yte = y[tr], y[te]

def perda(theta, X, alvo):
    return float(np.mean((X @ theta - alvo)**2))

def gradiente(theta, X, alvo):
    return (2 / len(alvo)) * X.T @ (X @ theta - alvo)

theta = np.array([0.1, -0.2])
h = 1e-6
aprox = np.array([(perda(theta + np.eye(2)[j]*h, Xtr, ytr) -
                   perda(theta - np.eye(2)[j]*h, Xtr, ytr))/(2*h) for j in range(2)])
assert np.allclose(aprox, gradiente(theta, Xtr, ytr), atol=1e-5)

historico = [perda(theta, Xtr, ytr)]
for _ in range(600):
    theta -= .1 * gradiente(theta, Xtr, ytr)
    historico.append(perda(theta, Xtr, ytr))
assert historico[-1] < historico[0]
print('Parâmetros estimados (intercepto, inclinação):', theta.round(3))
print('MSE inicial treino:', round(historico[0], 4))
```

```
print('MSE final treino:', round(historico[-1], 4))
print('MSE conjunto reservado:', round(perda(theta, Xte, yte), 4))
```

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Qual o gradiente de $(wx-y)^2$ em relação a w?

Resposta: $2(wx-y)x$; para uma média, calcular a média dos gradientes.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.

• Executar exemplos com pesos baixados sem registrar versão, licença e procedência.

• Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide